

1.项目介绍

1.1.车道线检测

车道线：用来管制和引导交通的一种标线，由标化于路面上的线条、箭头、文字、标记和轮廓标识组成；

常见的车道线检测方案：基于传统计算机视觉的检测、基于深度学习的道路特征检测、基于雷达等高精设备的车道线检测；

1.2.基于传统计算机视觉的车道线检测

（1）传统计算机视觉车道线检测主要依赖高度定义化的手工特征提取和启发式的方法；

（2）国内外广泛使用的检测方法：基于道路特征和道路模型两种方法；

（3）基于道路特征：利用车道线与道路之间的物理结构差异对图像进行后续的分割和处理，突出道路特征，实现车道线检测；

（4）基于道路模型：利用不同的道路图像模型(直线、抛物线、复合型)，对模型中的参数进行估计与确定，最终和车道线进行拟合；

1.2.1 基于道路特征检测

基于道路特征检测根据提取特征不同，分为：基于颜色特征、纹理特征、多特征融合；

1、基于颜色特征检测

（1）基于灰度特征检测

基于灰度特征检测：通过提取图像的灰度特征来检测道路边界和道路标识；

在车道图像中，路面与车道线交汇处的灰度值变换剧烈，利用边缘增强算子突出图像的局部边缘，定义像素的边缘强度，设置阈值方法提取边缘点；

常用的算子：Sobel 算子、Prewitt 算子、Log 算子、Canny 算子；

基于灰度特征检测结构简单，对于路面平整、车道线清晰的结构化道路尤为适用；但当光照强烈、有大量异物遮挡、道路结构复杂、车道线较为模糊时，检测效果受到很大的影响；

（2）基于彩色特征检测

基于彩色特征检测：通过提取图像的彩色特征来检测道路边界和道路标识，涉及颜色空间的选择和分割策略选取两方面；

颜色空间：由一组数值描述图像信息的抽象模型，常用的颜色空间：RGB 空间、HSL 空间、HSV 空间等；

2、 基于纹理特征检测

基于纹理特征检测：通过对包含多个像素点的区域中的纹理强度和纹理方向进行计算，从而对车道线进行检测；

基于纹理特征检测具有较强的抗噪能力，但当光照强度改变，图像分辨率改变时，计算结果出现偏差；二维图像中提取的纹理特征与三维物体实际的纹理有一定差别，一定程度上影响检测准确度；

1.2.2.基于道路模型检测

1、直线模型

直线模型建立在车道线为直线的假设基础上，直线模型数学表达式：

$$u=k(v-h)+b$$

其中： u 、 v 表示道路图像的横纵坐标， k 表示斜率， b 为截距， h 为道路消失线在途中的纵坐标； 其中： u 、 v 表示道路图像的横纵坐标， k 表示斜率， b 为截距， h 为道路消失线在途中的纵坐标；其中： u 、 v 表示道路图像的横纵坐标， k 表示斜率， b 为截距， h 为道路消失线在途中的纵坐标；

得到道路消失线水平位置后，只需得到 k 和 b 即可确定车道线在图像中的位置；在车辆行驶速度不高，且道路弯曲曲率不大的情况下，可以有较好的识别和导航效果；

2、双曲线模型

基于双曲线模型的车道线检测首先运用 Canny 算子对道路边缘进行检测；

采用 Hough 变换提取道路的边界点，使用扩展的 Kalman 滤波进行预测跟踪来减少道路扫描范围；

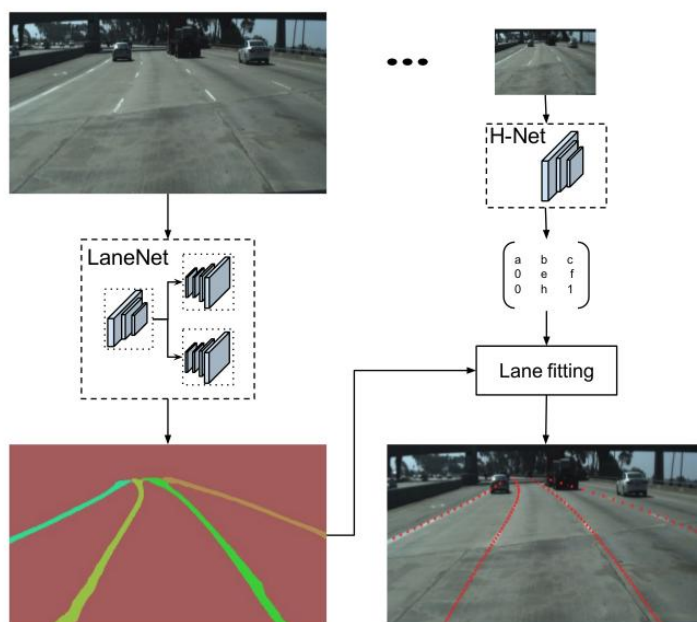
通过左右车道边界参数与双曲线模型参数进行匹配，利用最小二乘法来求解模型参数；

1.3 基于深度学习的车道线检测

1.3.1 LaneNet+H-Net 车道线检测

(1) LaneNet 将车道线检测问题转为实例分割问题，即：每个车道线形成独立的实例，但都属于车道线这一类别；H-Net 由卷积层和全连接层组成，利用转换矩阵 H 对同一车道线的像素点进行回归；

(2) 对于一张输入图片，LaneNet 负责输出实例分割结果，每条车道线一个标识 ID，H-Net 输出一个转换矩阵，对车道线像素点进行修正，并对修正后的结果拟合出一个三阶多项式作为预测的车道线；



CSDN @FUXI_Willard

(3) LaneNet 将实例分割拆分为语义分割和聚类两部分，编码器分为 Embedding 和 Segmentation 两个分支：

Embedding 负责对像素进行嵌入表示，训练得到嵌入向量进行聚类；

Segmentation 对输入的图像进行语义分割，并对像素点进行二分类，判断属于车道线还是背景；

1.3.2 SCNN 车道线检测

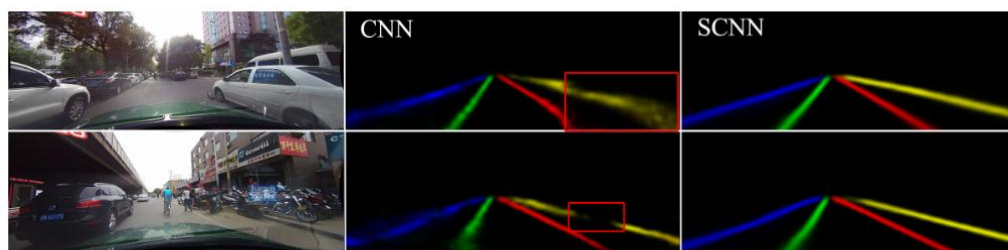
CNN 问题及 SCNN 特点：

(1) CNN 具有强大的特征提取能力，但由卷积块堆叠的 CNN 架构没有足够充分的探索图像行和列上的空间关系的能力；

(2) CNN 对于经常被遮挡的车道线识别效果不好；

(3) Spatial CNN(SCNN)将传统卷积层接层(layer-by-layer)的连接形式转为特征图中片连片卷积(slice-by-slice)的形式，使图像中的像素行和列之间可以传递信息；

(4) SCNN 对于长距离连续形状的目标、大型目标及有着极强空间关系但外观线索不明显的目标，如：车道线、电线杆等具有很好的检测效果；



图：CNN与SCNN对于车道线检测效果对比

CSDN @FUXI_Willard

1.4 基于激光雷达的车道线检测

基于激光雷达的车道线检测方法：基于反射强度信息

（1）基于激光雷达反射强度信息形成的灰度图，或根据强度信息与高程信息配合，过滤出无效信息；然后对车道线进行拟合；

（2）在激光雷达获取的道路环境的三维点云中，检测每一个激光层采集到的可行使区域的回波强度是否发生变化，如果发生变化，将变化点提取并进行标记；

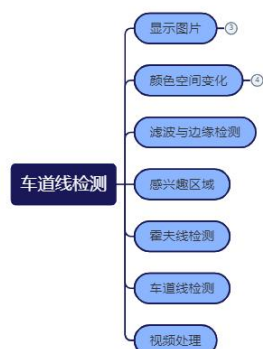
（3）通过对点云数据中有高程数据的点进行滤波，一定程度上确定出可行使区域，剔除一些和车道线的回波强度接近的物体；

（4）通过对提取的车道线点云进行聚类 and 去噪，利用最小二乘法进行拟合，最终提取出车道线；

不同物体的回波强度：

介质	回波强度/dBz	可能的物体分类
沥青、混凝土	5 ~ 8	道路、房屋等
特性涂层	12 ~ 30	车道线
植被、金属	45 ~ 150	树木、车辆等

1.5 基于 OpenCV 实现车道线检测



2.数据集

2.1.图片



2.2.视频



3.实验目标

- 1) 检测图像中车道线位置，将车道线信息提供路径规划。

4.实验环境

4.1.硬件环境要求

实验平台硬件实验环境：

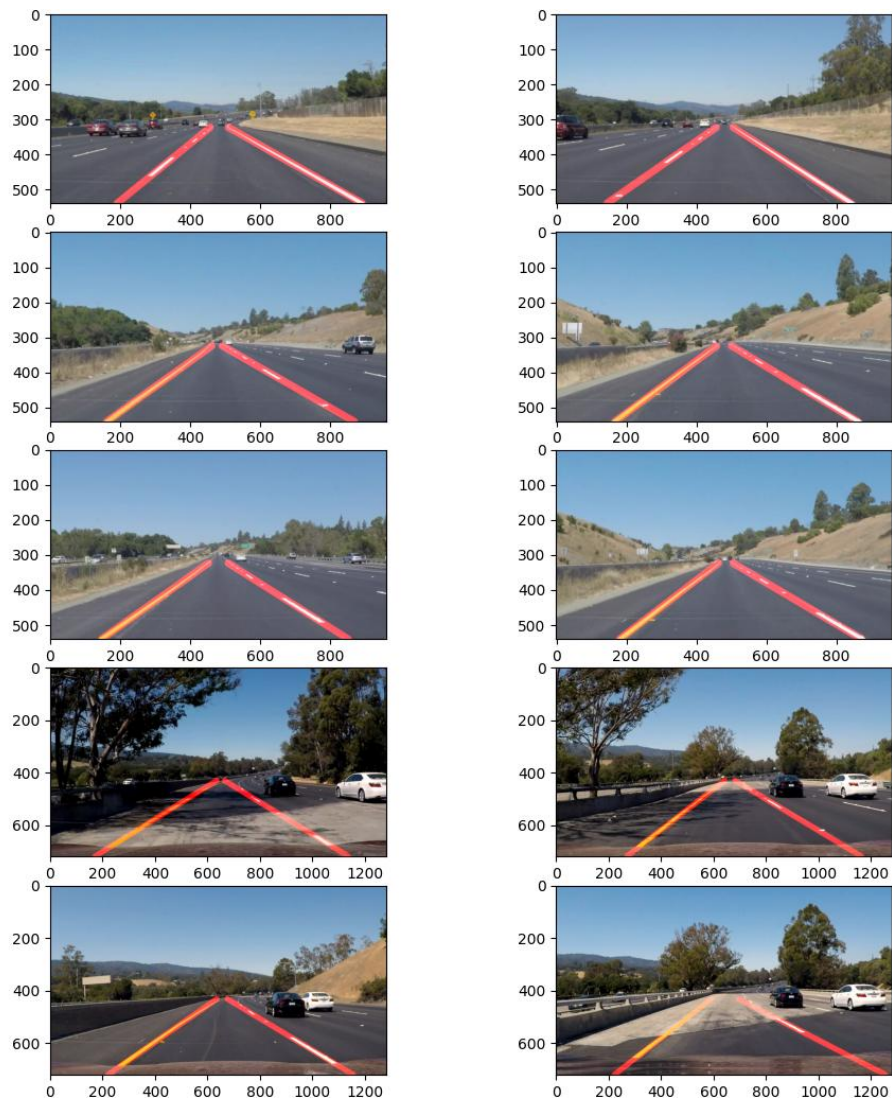
硬件	型号
CPU	Inter core i 7700k 多核
GPU	NVIDIA GTX1080TI 双卡

4.2.编程软件和版本

本实验在 Linux 操作系统下完成，需要涉及到以下软件（版本号仅供参考，可以使用不同版本）：

- 1) matplotlib;
- 2) opencv-python;
- 3) moviepy。

5.预期效果



6.实验步骤

6.1 显示图片

6.1.1 读取图像

`matplotlib.pyplot.imread(path)`

用于读取一张图片，将图像数据变成数组 `array`。

参数：

要读取的图像文件路径。

返回值：

如果是灰度图：返回(M,N)形状的数组，M表示高度，N表示宽度。

如果是 RGB 图像，返回(M, N, 3) 形状的数组，M表示高度，N表示宽度。

如果是 RGBA 图像，返回(M, N, 4) 形状的数组，M表示高度，N表示宽度。

此外，PNG 图像以浮点数组 (0-1) 的形式返回，所有其他格式都作为 `int` 型数组返回，位深由具体图像决定。

因此，在 `pytorch` 中如果需要读取文件，则需要通过 `.permute(2, 0, 1)` 将图像由 HWC->CHW

6.1.2 子图

`subplot(nrows, ncols, plot_number)`

或者

`subplot(nrows ncols plot_number)`也行（中间不用逗号，前提是只能是三位数）

这个函数用来表示把 `figure` 分成 `nrows*ncols` 的子图表示，

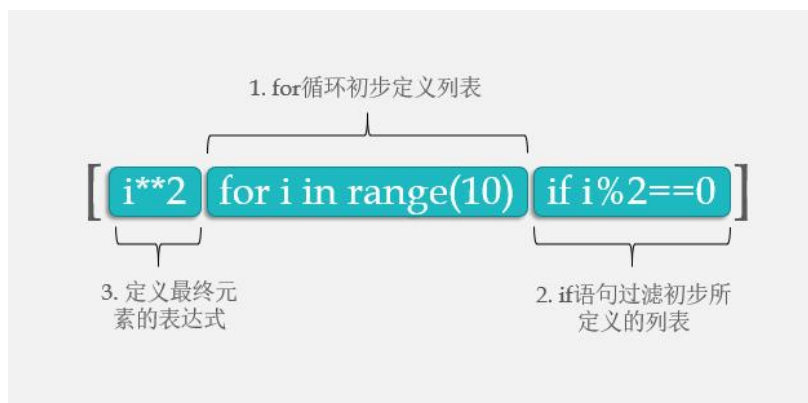
`nrows`：子图的行数

`ncols`：子图的列数

`plot_number` 索引值，表示把图画在第 `plot_number` 个位置（从左下角到右上角）

例如：`plt.subplot(2,3,1)`也可以简写 `plt.subplot(231)`表示把显示界面分割成 2*3 的网格。其中，第一个参数是行数，第二个参数是列数，第三个参数表示图形的标号。

6.1.3 列表推导式



列表推导式或者说列表解析式是一种非常简洁的创建列表的方式。

很多时候我们需要创建一个满足特定要求的新列表，不得不用 `for` 循环来创建，而用列表推导式来表达只需要一行代码即可。

列表推导式另一个优点是相比于 `for` 循环更高效，因为列表推导式在执行时调用的是 Python 的底层 C 代码，而 `for` 循环则是用 Python 代码来执行。

```
>>> vec = [-4, -2, 0, 2, 4]
```

```
>>>
```

```
>>> # 新生成一个列表，该列表中的每个元素是 vec 列表中每个元素的 2 倍
```

```
>>> [x*2 for x in vec]
```

```
[-8, -4, 0, 4, 8]
```

```
>>> # 利用 if 语句对列表进行过滤，将小于 0 的元素过滤掉
```

```
>>> [x for x in vec if x >= 0]
```

```
[0, 2, 4]
```


6.1.4 显示图像

```
8 import matplotlib.pyplot as plt
9 import cv2
0 import glob
1 import numpy as np
2 from moviepy.editor import VideoFileClip #处理视频

15 #定义一个显示图片的函数
16 def show_images(images, cmap=None):
17     cols = 2
18     rows = (len(images)+1) // cols #乘除 N行2列
19     plt.figure(figsize=(10,11)) #定义显示图
20     for i, image in enumerate(images):
21         plt.subplot(rows,cols, i+1) #子图像 (行。列。下标)
22         cmap='gray' if len(image.shape) == 2 else cmap
23         plt.imshow(image, cmap=cmap) #灰度或彩色显示
24         plt.xticks() #刻度
25         plt.yticks()
26     plt.tight_layout(pad=0, h_pad=0, w_pad=0) #紧凑显示
27     plt.show()

28
29 # 定义图像列表
30 test_images = [plt.imread(path) for path in glob.glob("test_images/*.jpg")]
31 # 显示原图
32 show_images(test_images)
33
```

6.2 RGB 颜色空间

6.2.1 设阈值，去除背景部分

cv2.inRange()

例如

mask = cv2.inRange(rgb, lower_red, upper_red)

函数很简单，参数有三个

第一个参数：rgb 指的是原图

第二个参数：lower_red 指的是图像中低于这个 lower_red 的值，图像值变为 0

第三个参数: `upper_red` 指的是图像中高于这个 `upper_red` 的值, 图像值变为 0
而在 `lower_red~upper_red` 之间的值变成 255

6.2.2 位运算

(1)与运算 `cv2.bitwise_and`

`cv2.bitwise_and(src1, src2, dst=None, mask=None)`

`src1`: 参与与运算的图像

`src2`: 用 `src2` 与 `src1` 进行与运算

`dst`: 与输入具有相同大小和类型的输出数组

`mask`: 可选操作掩码, 8 位单通道数组, 即指定要更改的输出数组的元素。

与运算操作就是 $1 \& 1 = 1$, 其他为 0。

(2)或运算 `cv2.bitwise_or`

`cv2.bitwise_or(src1, src2, dst=None, mask=None)`

或运算 $0 \mid 0 = 0$, 其他为 1

6.2.3 对图片进行特定颜色过滤 (RGB 颜色空间)

```
34
35 # RGB颜色空间
36 def select_rgb_white_yellow(image):
37     lower = np.uint8([200, 200, 200])
38     upper = np.uint8([255, 255, 255])
39     white_mask = cv2.inRange(image, lower, upper)
40     lower = np.uint8([190, 190, 0])
41     upper = np.uint8([255, 255, 255])
42     yellow_mask = cv2.inRange(image, lower, upper)
43     # 合并
44     mask = cv2.bitwise_or(white_mask, yellow_mask)
45     # 提取白色与黄色
46     masked = cv2.bitwise_and(image, image, mask=mask)
47     return masked
48
```

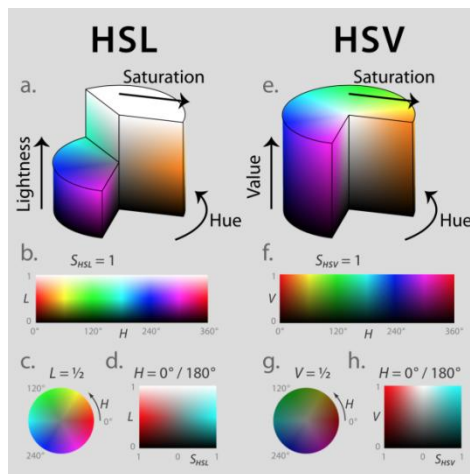
```

49 show_images(list(map(select_rgb_white_yellow, test_images)))
50 # test_images :十张图片的列表
51 # select_rgb_white_yellow() 是个函数
52 # map(函数, 列表) —> 用这个作用在列表的每个值上
53 # list(map) 得到新的处理结果列表
54 #for image in test_images:
55 #     select_rgb_white_yellow(image)
56

```

6.3 HSL 颜色空间

6.3.1 颜色空间



6.3.2 颜色空间变换

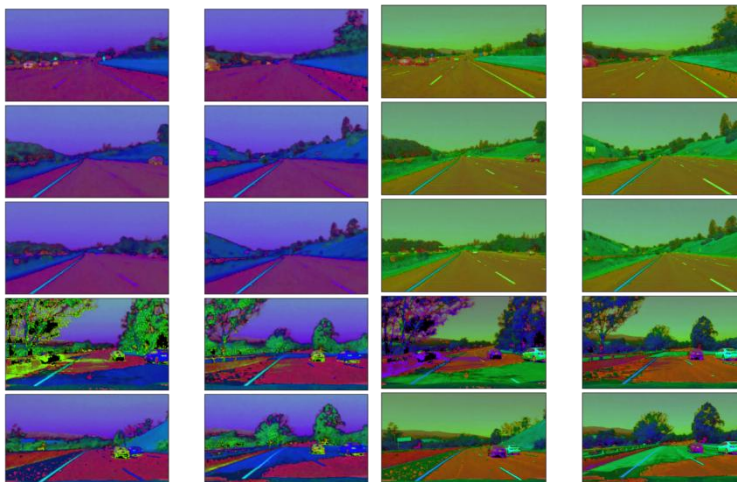
cv2.cvtColor(frame,code)

COLOR_BGR2BGRA,COLOR_RGB2BGRA	↘
COLOR_BGRA2RGBA,COLOR_BGR2BGRA	↘
COLOR_BGRA2BGR	
COLOR_RGB2GRAY,COLOR_GRAY2RGB	↘
COLOR_RGBA2GRAY,COLOR_GRAY2RGBA	
COLOR_BGR2HSV、COLOR_RGB2HSV、 COLOR_HSV2BGR、COLOR_HSV2RGB	
COLOR_RGB2YCrCb,COLOR_RGB2YCrCb, COLOR_YCrCb2BGR、COLOR_YCrCb2RGB (可以用 YUV 代替 YCrCb)	
COLOR_BGR2XYZ,COLOR_RGB2XYZ, COLOR_XYZ2BGR, COLOR_XYZ2RGB	
COLOR_BGR2HLS,COLOR_RGB2HLS, COLOR_HLS2BGR, COLOR_HLS2RGB	
COLOR_BGR2Lab,COLOR_RGB2Lab, COLOR_Lab2BGR, COLOR_Lab2RGB	
COLOR_BGR2Luv,COLOR_RGB2Luv, COLOR_Luv2BGR, COLOR_Luv2RGB	

```

57 # HSV颜色空间
58 def convert_hsv(image):
59     return cv2.cvtColor(image, cv2.COLOR_RGB2HSV)
60
61 show_images(list(map(convert_hsv, test_images)))
62
63 #HSL颜色空间    --黄色和白线都可显示
64 def convert_hsl(image):
65     return cv2.cvtColor(image, cv2.COLOR_RGB2HLS)
66
67 show_images(list(map(convert_hsl, test_images)))
68

```



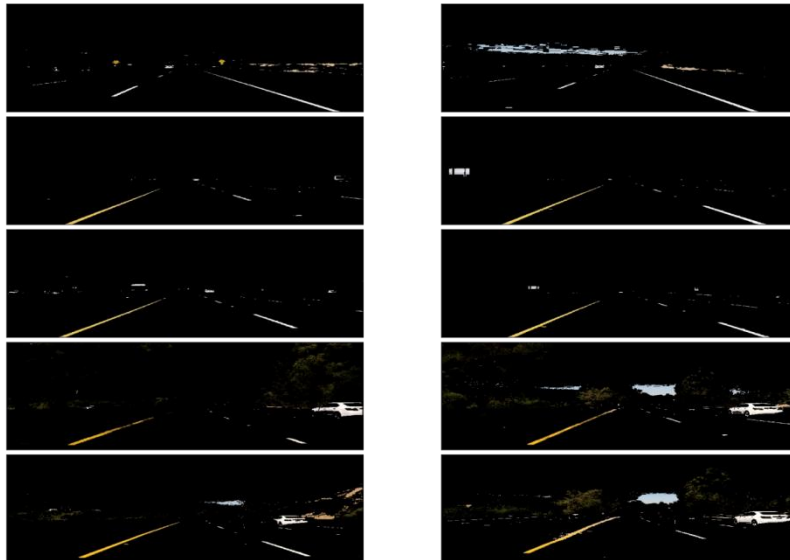
HSV 与 HSL 对比

用 HSL 比用 RGB 和 HSV 更容易显出图片中白线和黄线。

6.3.3 对图片进行特定颜色过滤(HSL 颜色空间)

```
69 # 使用HSL颜色空间对图片进行特定颜色过滤
70 def select_hsl_white_yellow(image):
71     hsl = convert_hsl(image)
72     lower = np.uint8([0, 200, 0])
73     upper = np.uint8([255, 255, 255])
74     white_mask = cv2.inRange(hsl, lower, upper)
75     lower = np.uint8([10, 0, 100])
76     upper = np.uint8([40, 255, 255])
77     yellow_mask = cv2.inRange(hsl, lower, upper)
78     # 合并二值图
79     mask = cv2.bitwise_or(white_mask, yellow_mask)
80     # 二值图与原图进行按位与，得到黄线与白线
81     masked = cv2.bitwise_and(image, image, mask=mask)
82     return masked
83
84
85 # 保存HSL检测结果
86 white_yello_images = list(map(select_hsl_white_yellow, test_images))
87 show_images(white_yello_images)
88
```

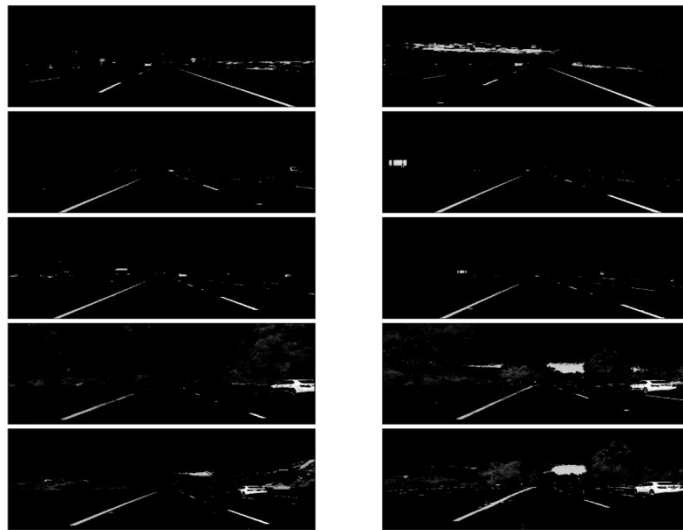
对 HSL 图的颜色空间进行黄白色线的特定颜色过滤之后：



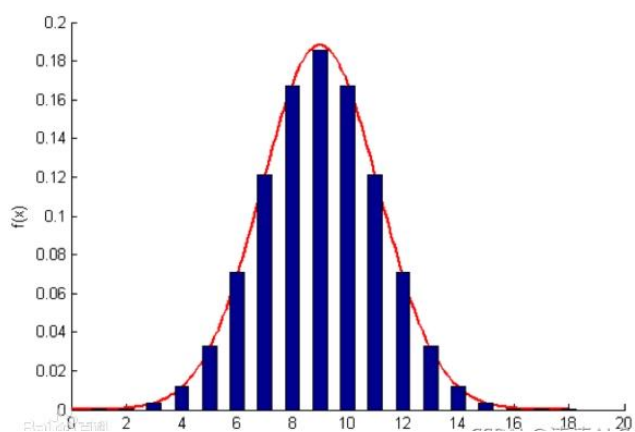
6.4 边缘检测

6.4.1 灰度化

```
89
90 #灰度化
91 def convert_gray_scale(image):
92     return cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
93
94 gray_images = list(map(convert_gray_scale, white_yello_images))
95 #show_images(gray_images)
96
```



6.4.2 滤波去噪



2	4	5	4	2
4	9	12	9	4
5	12	15	12	5
4	9	12	9	4
2	4	5	4	2

高斯滤波是一种线性平滑滤波

高斯滤波是对整幅图像进行加权平均的过程，每一个像素点的值都由其本身和邻域内的其他像素值经过加权平均后得到。高斯滤波的具体操作是：用一个模板（或称卷积、掩模）扫描图像中的每一个像素，用模板确定的邻域内像素的加权平均灰度值去替代模板中心像素点的值。

基于二维高斯函数，构建权重矩阵，进而构建高斯核，最终对每个像素点进行滤波处理（平滑、去噪）

`dst=cv2.GaussianBlur (src,ksize,sigmaX,sigmaY,borderType)`

`dst` 是返回值，表示进行高斯滤波后得到的处理结果。

- `src` 是需要处理的图像，即原始图像。它能够有任意数量的通道，并能对各个通道 独立处理。图像深度应该是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F` 或者 `CV_64F` 中的一种。

- `ksize` 是滤波核的大小。滤波核大小是指在滤波处理过程中其邻域图像的高度和宽 度。需要注意，滤波核的值必须是奇数。

- `sigmaX` 是卷积核在水平方向上（X 轴方向）的标准差，其控制的是权重比例。

- `sigmaY` 是卷积核在垂直方向上（Y 轴方向）的标准差。如果将该值设置为 0，则只采用 `sigmaX` 的值

如果 `sigmaX` 和 `sigmaY` 都是 0，则通过 `ksize.width` 和 `ksize.height` 计算得到。其中：

$$\text{sigmaX}=0.3\times[(\text{ksize.width}-1)\times0.5-1]+0.8$$

$$\text{sigmaY}=0.3\times[(\text{ksize.height}-1)\times0.5-1]+0.8$$

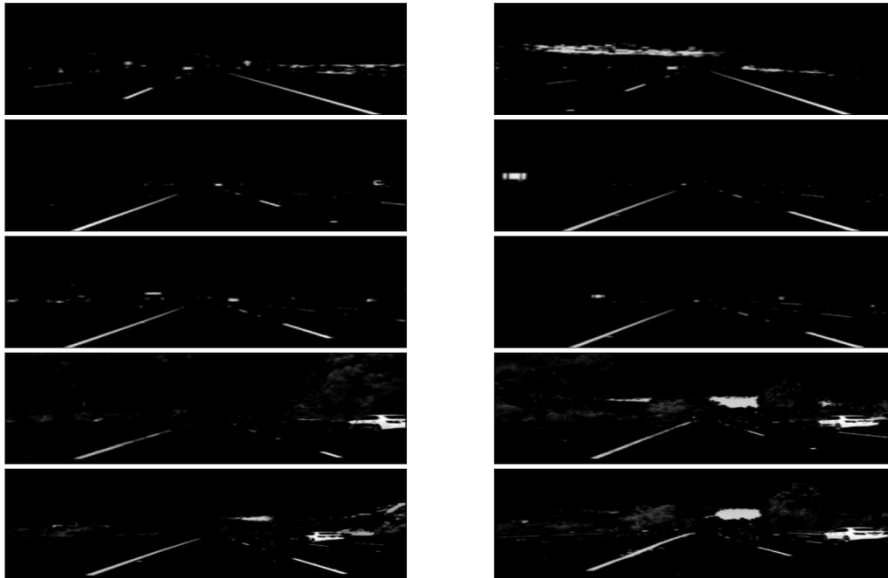
- `borderType` 是边界样式，该值决定了以何种方式处理边界。一般情况下，不需要考虑该值，直接采用默认值即可。 在该函数中，`sigmaY` 和 `borderType` 是可选参数。`sigmaX` 是必选参数，但是可以将该参数设置为 0，让函数自己去计算 `sigmaX` 的具体值。

滤波去噪是为了去除图片中多余的干扰白点，使得这些白点不会影响到车道线的检测

```

97 # 滤波去噪
98 def apply_smoothing(image, ksize=15):
99     return cv2.GaussianBlur(image, (ksize,ksize), 0)
100
101 blurred_images = list(map(apply_smoothing, gray_images))
102 show_images(blurred_images)

```



6.4.3 Canny 边缘检测

Canny 边缘检测分为如下几个步骤：

- (1)应用高斯滤波平滑图像，目的是去除噪声。
- (2)计算图像梯度，得到可能边缘；
- (3)应用非最大抑制技术来消除边误检；
- (4)应用双阈值的方法筛选边缘信息；
- (5)利用滞后技术来跟踪边界。

```
edge = cv2.Canny(image, threshold1, threshold2[, edges[, apertureSize[, L2gradient ]]])
```

参数含义如下：

image: 要检测的图像

threshold1: 阈值 1（最小值）

threshold2: 阈值 2（最大值），使用此参数进行明显的边缘检测

edges: 图像边缘信息

apertureSize: sobel 算子（卷积核）大小

L2gradient : 布尔值。

True: 使用更精确的 L2 范数进行计算（即两个方向的导数的平方和再开方）

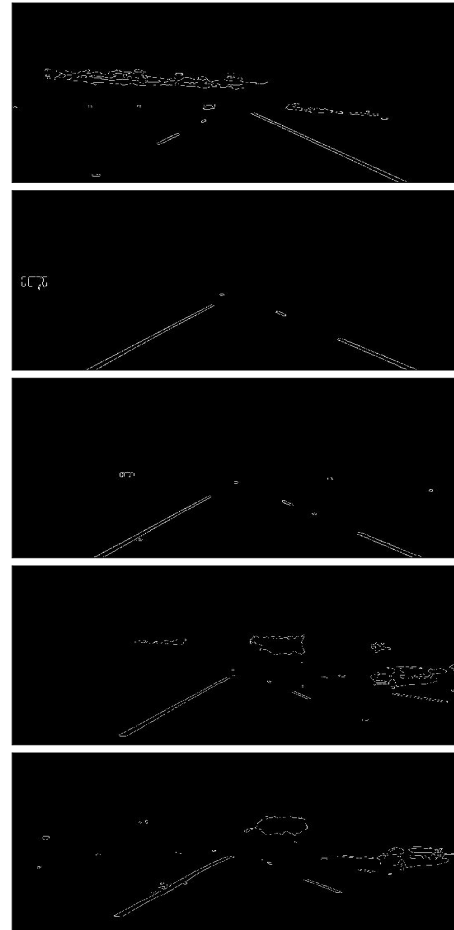
False: 使用 L1 范数（直接将两个方向导数的绝对值相加）

进行边缘检测能更有效地找出图中的车道线


```

103
104 #边缘检测
105 def detect_edges(image, low_thresh=50, high_thresh=150):
106     return cv2.Canny(image, low_thresh, high_thresh)
107
108 edge_images = list(map(detect_edges, blurred_images))
109 show_images(edge_images)
110

```



6.5 感兴趣区域

6.5.1 填充多边形



以填充矩形为例，想填充以 1、2、3、4 为顶点的矩形，我就要按 1、2、3、4 的顺序给出坐标序列 array，或者是连贯的相邻顶点顺序（比如 1432，4321...）

函数原型：

`cv2.fillPoly(image, ppt, Scalar(255,255,255))`

参数：

image: 表示的是多边形将被画到 image 上

ppt: 表示的是多边形的顶点集为 ppt

Scalar: 表示的是多边形的颜色定义为 `Scalar(255,255,255)`，即 RGB 的值为白色

```
112 # 多边形填充+按位与过滤
113 def filter_region(image, vertices):
114     mask = np.zeros_like(image) # 全0图像，大小与image相同
115     if len(mask.shape) == 2: # 灰度图，只有宽高，通道数默认为1
116         cv2.fillPoly(mask, vertices, 255)
117     else: # 多通道图像
118         cv2.fillPoly(mask, vertices, (255,)*mask.shape[2])
119     return cv2.bitwise_and(image, mask)
120
```

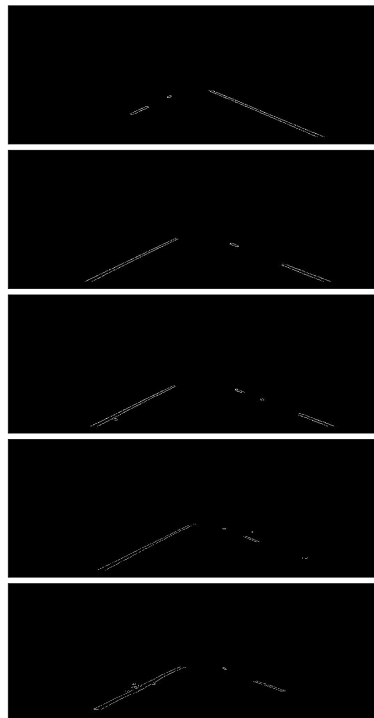
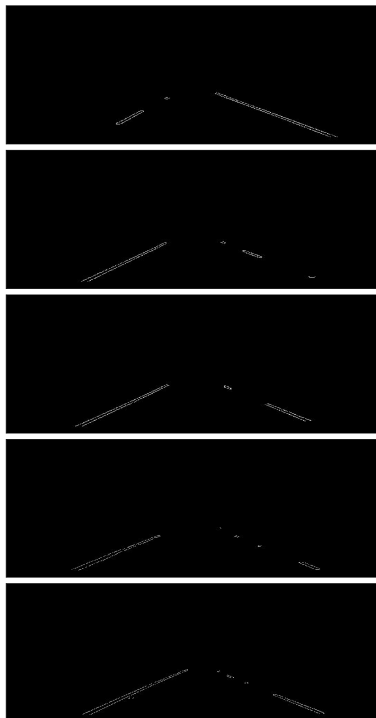
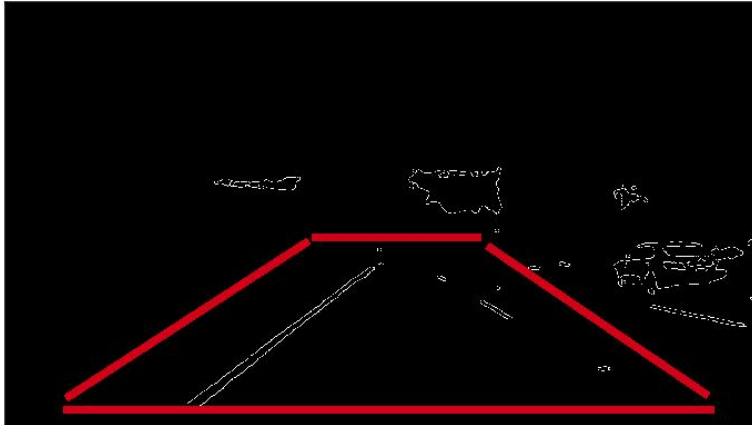
实现对图像的 roi 掩膜处理

6.5.2 感兴趣区域范围

```

121 # 感兴趣区域 Region of Interest => ROI
122 def select_region(image):
123     rows, cols = image.shape[0:2] #得到宽高
124     bottom_left = [cols * 0.1, rows * 0.95] #左上、左下, 右上、右下
125     top_left = [cols * 0.4, rows * 0.6]
126     bottom_right = [cols * 0.9, rows * 0.95]
127     top_right = [cols * 0.6, rows * 0.6]
128     #定义一个numpy数组, 表示选中的区域
129     vertices = np.array([[bottom_left, top_left, top_right, bottom_right]], dtype=np.int32)
130     # 保留选中区域像素值, 非感兴趣区域变成0
131     return filter_region(image, vertices)
132
133 roi_images = list(map(select_region, edge_images))
134 show_images(roi_images)
135

```

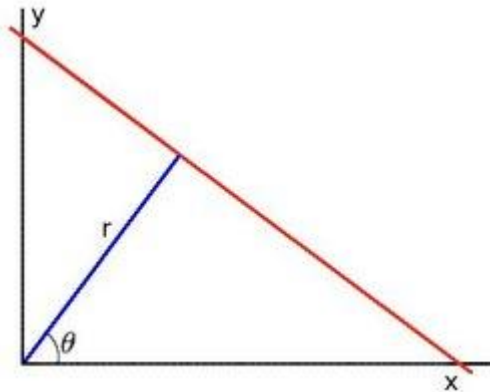


6.6 霍夫线检测

6.6.1 原理

霍夫直线检测过程，是将笛卡尔坐标系下的直线方程转换到极坐标系下，并根据各个像素的坐标点代入得到的方程，获取对应的直线参数。

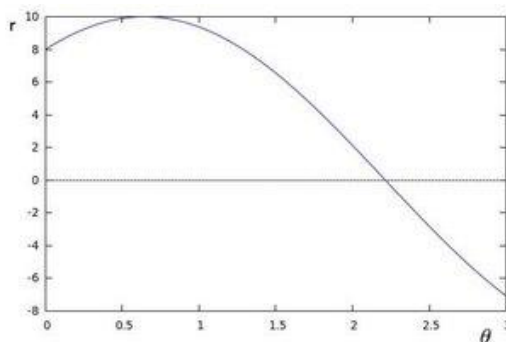
(1) 直线在笛卡尔坐标系下的方程为 $y=kx+b$,



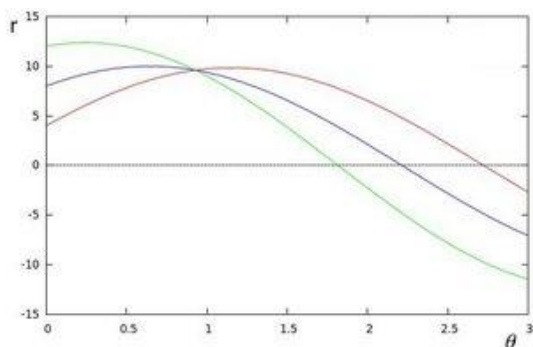
r 为原点距离直线的欧氏距离， θ 是线段 r 与 x 轴的夹角。转换到极坐标系的方程为：

$$\begin{cases} x = r * \cos \theta \\ y = r * \sin \theta \end{cases} \Rightarrow r = x * \cos \theta + y * \sin \theta$$

将图像像素点的坐标，代入方程，可得相应的曲线。如有一个位置下标为 $(8, 6)$ ，则输出的曲线为

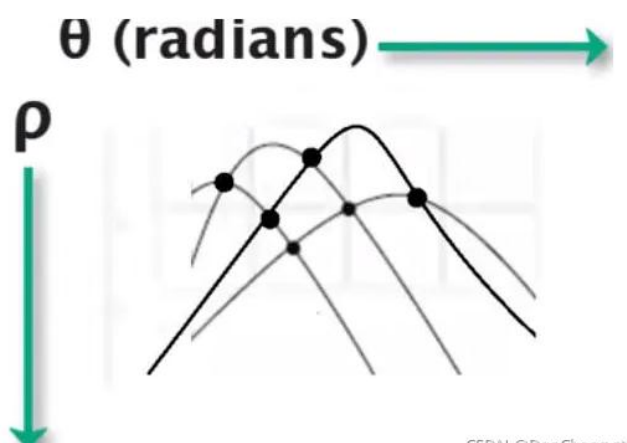


通过图像的宽度 W 和高度 H ，对应的位置坐标，将其代入方程，可得到 $W*H$ 条曲线，曲线的交点处，对应直线经过点的个数。

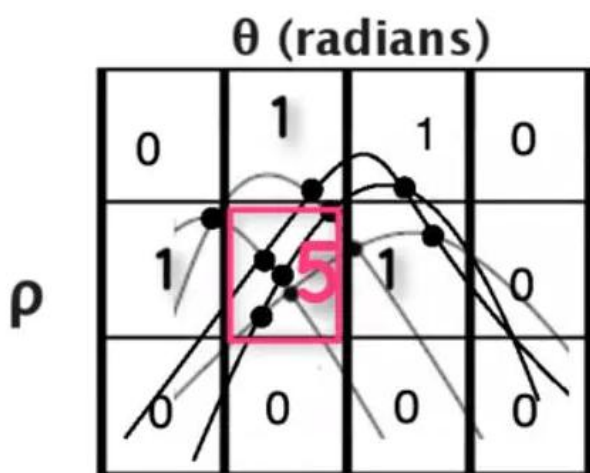


该曲线在霍夫空间中，每过一个点，对应的位置像素加 1，那么像素越高，越亮的那个点，说明对应的直线过的点越多。

当有很多点在霍夫空间的曲线相交于一点时候，就说明这些点具有相同的 θ 与 r ，即它们都属于同一条直线，而参数 θ 与 r 就是该直线在霍夫空间的直线参数方程。



通过下图划分小格，只要曲线穿过就会对小格进行投票，我们记录投票数量，记录最多的作为参数



(2) 已知最亮的点，即已知极坐标中的 θ 和 r ，反算到笛卡尔坐标系下，可得到关于 x , y 的方程。输入不同的 x ，得到不同的 y ，可在图像中绘制出一条直线。

$$y = \left(-\frac{\cos \theta}{\sin \theta} \right) x + \left(\frac{r}{\sin \theta} \right)$$

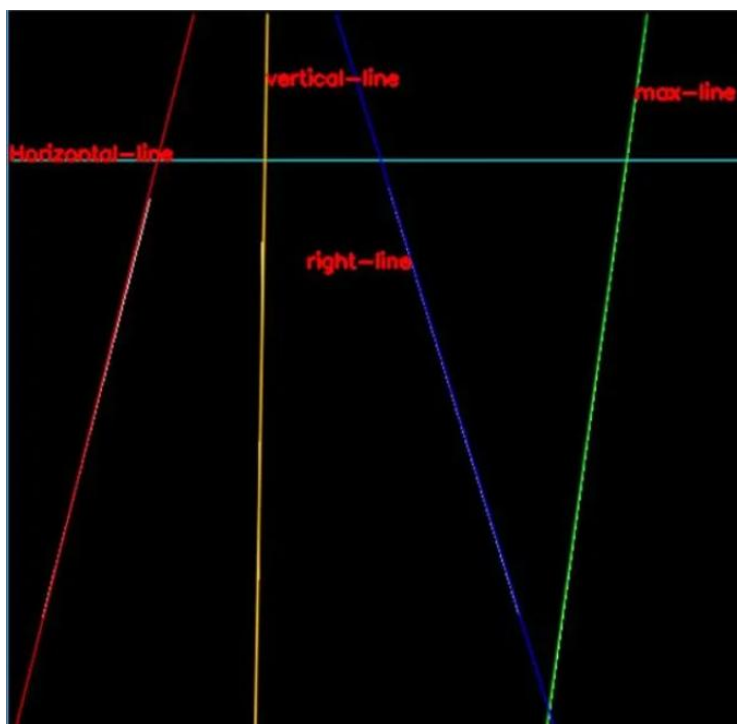
(3) 通过标准霍夫直线变换实现如下直线分类与最大直线提取

左侧倾斜直线

右侧倾斜直线

水平或者垂直线

长度最大直线



6.6.2 二值图像中查找直线

函数原型

```
HoughLinesP(image,
              rho, theta, threshold, lines=None, minLineLength=None, maxLineGap=None)
```

参数:

image: 必须是二值图像，推荐使用 canny 边缘检测的结果图像；

rho 参数表示参数极径 以像素值为单位的分辨率，这里一般使用 1 像素。

theta 参数表示参数极角 以弧度为单位的分辨率，这里使用 1 度($\text{numpy.pi}/180$)。

threshold 参数表示检测一条直线所需最少的曲线交点。

lines 参数表示储存着检测到的直线的参数对的容器，也就是线段两个端点的坐标。

minLineLength 参数表示能组成一条直线的最少点的数量，点数量不足的直线将被抛弃。

maxLineGap 参数表示能被认为在一条直线上的亮点的最大距离。

同一方向上两条线段判定为一条线段的最大允许间隔（断裂），超过了设定值，则把两条线段当成一条线段，值越大，允许线段上的断裂越大，越有可能检出潜在的直线段

返回值：

该函数能输出检测到的直线的端点（x0,y0,x1,y1）

```
137 #霍夫线检测(直线)
138 def hough_lines(image):
139     return cv2.HoughLinesP(image, rho=1, theta=np.pi/180, threshold=20, minLineLength=20, maxLineGap=300)
140
141 list_of_lines = list(map(hough_lines, roi_images))
142
```

6.6.3 绘制直线

```
143
144 #绘制直线
145 def draw_lines(image, lines, color=(255,0,0), thickness=2, cp=True):
146     if cp:
147         image = np.copy(image) # 在副本中画图，而不会修改原图
148     for line in lines:
149         for x1,y1,x2,y2 in line:
150             cv2.line(image, (x1,y1), (x2,y2), color, thickness)
151     return image
152
153 for i in range(len(test_images)):
154     draw_lines(test_images[i], list_of_lines[i])
155
156 line_images = [] # 原图RGB 线段坐标
157 for image, lines in zip(test_images, list_of_lines):
158     line_images.append(draw_lines(image, lines))
159
160 show_images(line_images)
161
```

6.7 车道线检测

6.7.1 车道线计算

剔除离群点

利用斜率的合理性，剔除误差较大的点

合理性判断

在确定车道线之前，对直线进行合理性判断，因为车道线不会突变，所以主要是利用上一帧的斜率以及大致斜率来排除


```

161
162 # 车道线合理性判断
163 def average_slope_intercept(lines):
164     left_lines = [] # (slope, intercept) 斜率、截距  $y = ax + b$ 
165     left_weights = [] # (length,)
166     right_lines = [] # (slope, intercept)
167     right_weights = [] # (length,)
168
169     for line in lines:
170         for x1, y1, x2, y2 in line:
171             if x2 == x1: # 忽略垂直线
172                 continue
173             slope = (y2 - y1) / (x2 - x1) #  $\tan\theta = y/x = (y2-y1)/(x2-x1)$ 
174             intercept = y1 - slope * x1
175             length = np.sqrt((y2 - y1) ** 2 + (x2 - x1) ** 2) # 两点之间的距离
176             if slope < 0: # y is reversed in image
177                 left_lines.append((slope, intercept))
178                 left_weights.append((length))
179             else:
180                 right_lines.append((slope, intercept))
181                 right_weights.append((length))
182
183     # add more weight to longer lines
184     left_lane = np.dot(left_weights, left_lines) / np.sum(left_weights) if len(left_weights) > 0 else None
185     right_lane = np.dot(right_weights, right_lines) / np.sum(right_weights) if len(right_weights) > 0 else None
186
187     return left_lane, right_lane # (slope, intercept), (slope, intercept)
188

```

6.7.2 车道线检测函数

```

188
189 # 确保直线端点坐标为整型
190 def make_line_points(y1, y2, line):
191     """
192     Convert a line represented in slope and intercept into pixel points
193     """
194     if line is None:
195         return None
196
197     slope, intercept = line
198
199     # 确保坐标点为整型值, 因为 cv2.line 需要
200     x1 = int((y1 - intercept) / slope)
201     x2 = int((y2 - intercept) / slope)
202     y1 = int(y1)
203     y2 = int(y2)
204
205     return ((x1, y1), (x2, y2))
206

```

```

207 # 车道线检测函数-根据直线的斜率延长车道线的长度
208 def lane_lines(image, lines):
209     left_lane, right_lane = average_slope_intercept(lines)
210     y1 = image.shape[0] # bottom of image
211     y2 = y1 * 0.6
212     # 转换成整型值的坐标
213     left_line = make_line_points(y1, y2, left_lane)
214     right_line = make_line_points(y1, y2, right_lane)
215
216     return left_line, right_line # 延长后直线的新的端点坐标
217

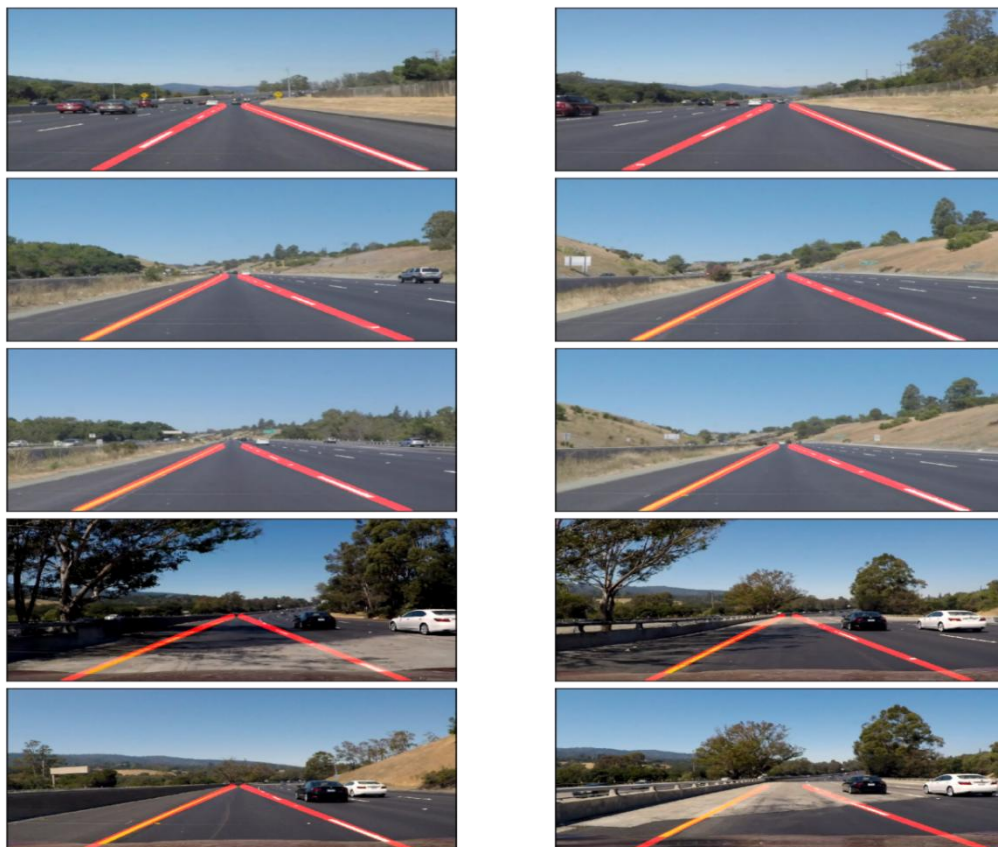
```

6.7.3 绘制车道线函数

```
217
218 # 绘制车道线
219 def draw_lane_lines(image, lines, color=(255,0,0), thickness=20):
220     line_image = np.zeros_like(image)
221     for line in lines:
222         if line is not None: # *line => *(left_line, right_line)
223             cv2.line(line_image, *line, color, thickness)
224     return cv2.addWeighted(image, 1.0, line_image, 0.95, 0)
225
```

6.7.4 车道线检测并绘制

```
225
226 lane_images=[]
227 for image, lines in zip(test_images, list_of_lines):
228     lane_images.append(draw_lane_lines(image, lane_lines(image, lines)))
229
230 show_images(lane_images)
231
```



6.8 视频处理

6.8.1 车道线检测流程

```
232 # 车道线检测流程
233 def LaneLine_process(image):
234     test_image = image #取一张图片
235     white_yello_images = select_hsl_white_yellow(test_image) #取黄白部分
236     gray_images = convert_gray_scale(white_yello_images) #灰度化
237     blurred_images = apply_smooththing(gray_images) #滤波去噪
238     edge_images = detect_edges(blurred_images) #边缘检测
239     roi_images = select_region(edge_images) #获取感兴趣区域
240     list_of_lines = hough_lines(roi_images) #获得车道线坐标
241     #绘制车道线
242     lane_images = (draw_lane_lines(test_image, lane_lines(test_image, list_of_lines)))
243     return lane_images
244
```

6.8.2 moviepy 库

MoviePy 是一个用于视频编辑的 Python 模块，它可被用于一些基本操作（如剪切、拼接、插入标题）、视频合成（即非线性编辑）、视频处理和创建高级特效。

安装

```
pip install moviepy
```

参考文档：

https://blog.csdn.net/qq_42691298/article/details/126369334

导入库：from moviepy.editor import VideoFileClip

使用 VideoFileClip 类：

```
__init__(self, filename, has_mask=False,
          audio=True, audio_buffersize=200000,
          target_resolution=None, resize_algorithm='bicubic',
          audio_fps=44100, audio_nbytes=2, verbose=False,
          fps_source='tbr')
```

filename: 视频文件名 .ogv, .mp4, .mpeg, .avi, .mov 等格式都可以

has_mask: 如果视频文件中包含遮罩该参数设置为“True”

audio: 如果视频没有声音或者希望加载时不读取音频，则应该将 audio 设置为 False

audio_buffersize: 音频缓冲区大小，一般使用缺省值即可

target_resolution: 设置为加载后需要变换到的分辨率，类型为列表或元组，第一个元素为分辨率的高，第二个为宽，如果高或宽有一个为 None，则保持现有纵横比调整帧的大小。

resize_algorithm: 要改变加载后的视频分辨率, 可以通过 **resize_algorithm** 指定调整分辨率的算法, 缺省值为 “bicubic”, 还可以是 “bilinear” 、 “fast_bilinear” 等。

audio_fps: 声音的采样频率

audio_nbytes: 采样的位数

verbose: 是否在标准输出设备上显示处理信息

fps_source: 从视频的元数据 **metadata** 哪个数据中获取 **fps** 值, 默认设置为 ‘tbr’, 但可以设置为 ‘fps’, 这可能有助于导入慢动作视频, 否则会弄糟。

6.8.3 输出处理结果

```
244
245 #视频输出存储
246 output_1 = 'video_1_xlt.mp4'
247 output_2 = 'video_2_xlt.mp4'
248 output_3 = 'video_3_xlt.mp4'
249
250 clip_1 = VideoFileClip(r"test_videos\challenge.mp4") # input video
251 out_clip_1 = clip_1.fl_image(LaneLine_process) #对视频的每一帧进行处理
252 out_clip_1.write_videofile(output_1, audio=True) #将处理后的视频写入新的视频文件
253
254 clip_2 = VideoFileClip(r"test_videos\solidWhiteRight.mp4") #input video
255 out_clip_2 = clip_2.fl_image(LaneLine_process) #对视频的每一帧进行处理
256 out_clip_2.write_videofile(output_2, audio=True) #将处理后的视频写入新的视频文件
257
258 clip_3 = VideoFileClip(r"test_videos\solidYellowLeft.mp4") #input video
259 out_clip_3 = clip_3.fl_image(LaneLine_process) #对视频的每一帧进行处理
260 out_clip_3.write_videofile(output_3, audio=True) #将处理后的视频写入新的视频文件
```